



CNG 465

INTRODUCTION TO BIOINFORMATICS

Fall 25-26

Semest Project Report

Zanyar kozan 2584985

ABSTRACT

The primary goal of the project was to design and develop a family tree and to design algorithms implementing family relationships based on this design. It is a model of individuals, relationships of the parent and child, and relationships of marriage.

The project was written in Python programming language using object orientation. The basic classes were deployed to handle persons, gender-specific persons, partnerships and traversal nodes. Then different functions were built on the developed structure based on the relationships within the families. This includes parents, kids, ancestors, descendants and other relationships through various graph methods by traversals.

The developed system makes the traversal of the family relationship in a systematic method with the help of breadth-first search algorithm. To ensure the correct values of the developed system has been confirmed through various simulations performed on different family structures. The simulation results indicate that the developed system is capable of displaying the family relationships in a proper manner and performing genealogical queries effectively.

Table of Contents

1. ABSTRACT 2

Table of Contents 3

Table of Figures 4

2. INTRODUCTION 5

3. CONTRIBUTION 5

4. PROBLEM STATEMENT 6

5. SOLUTION 7

1) Partnership Class 7

A) __str__ Function 8

B) __repr__ Function 9

C) is_parent Function 10

D) is_mother Function 11

E) is_father Function 12

F) add_parent Function 13

G) add_child Function 14

H) add_children Function 15

I) traverse Function 15

2) Person Class 21

A) __str__ Function 22

B) __eq__ Function 23

C) ancestor Function 24

D) descendant Function 26

6. RESULTS 28

7. DATA FLOW DIAGRAM 29

8. CONCLUSION 30

9. REFERENCES 30

Table of Figures

Figure 1	__str__ Function at Partn ship class	8
Figure 2	__repr__ Function at Partn ship class	9
Figure 3	is_parent Function at Partn ship class	10
Figure 4	is_moth Function at Partn ships class	11
Figure 5	is_fath Function at Partn ship class	12
Figure 6	add_parent Function at Partn ship class	13
Figure 7	add_child Function at Partn ship class	14
Figure 8	add_children Function at Partn ship class	15
Figure 9	trav se Function at Partn ship class	18
Figure 10	__str__ Function at P son class	22
Figure 11	__eq__ Function at P son class	23
Figure 12	ancestor Function at P son class	24
Figure 13	descendants Function at P son class	26
Figure 14	Test Results	28
Figure 15	Data Flow Diagram	29

INTRODUCTION

The objective of this Project was to develop a family tree system with the capability of organizing family relations in an organized way. The use of family trees are common in applications concerning bioinformatics as there is a need to display the relationships of the various people efficiently.

In the Project a family relationship network, a graph structure was created using object oriented programming with Python. Members of a family and their relationship information including gender relationships and parent and child relationships were clearly represented using objects. Functions were also created using a graph structure including parents, children and other relationships of a person depending on the graph traversal methods.

Not only was the aim of the project to store family data but also to analyze the relations systematically which means traversal algorithms. By doing so the system is able to analyze both the design process implementations and testing the code of the developed system.

CONTRIBUTION

In this Project, () and () was involved mainly in coding implementation and testing.

Drawing the Project's logic algorithm flowchart Zanyar kozan(2584985) was involved.

In the Report Writing, translating functions to LaTeX format, describing the system design and functionality and documenting the system, () and () was involved.

PROBLEM STATEMENT

The primary problem that is handled in the project is the fact that there is no efficient software to handle relationships between families and to traverse through the relationships represented in the software. This is because the easiest way to represent these relationships between the families will only involve the parents or the children, but it is not effective when relationships are to be queried both up and downward.

Another problem is related to ensuring consistency while traversing the family structure. When traversing the tree, it is necessary that the persons being traversed are not traversed more than once. To add to this, it is required for it to process the relationships in the correct way in the same framework for the relationships like ancestors, descendants, siblings and cousins.

Thus, the problem statement can be conceptualized as encapsulating the design of a family tree system such that: correctly reflects persons, sexes, partnerships and relationships of parents and children, provides support for systematic traversal of the family structure, simplifies looking into multiple family relationships in one common manner and is also scalable for additional analysis tasks.

This project proposes the resolution of these problems by modeling the family relations using a representation involving graphs and by building functions using traversals to examine said relationships.

SOLUTION

To solve the problem that was defined a graph model of the family structure was developed using object oriented programming techniques and Python programming.

The rationale was that people and their relationships could be created explicitly and these relationships could then be evaluated using formal graph traversal techniques.

In this solution persons, partnerships, parent and child relationships were modeled using distinct classes. Thus this was beneficial in making sure that the system is able to differentiate between persons and their relationships. According to this methods were developed that allow traversal through the family tree.

Every key function in the system traces an algorithm sequence clearly. Thus in this respect the algorithms used in the function implementation will be represented in the formal algorithm description style by applying the use of the LaTeX format as used in the standard algorithm definition in formal literature.

In the following subsections, the primary algorithms employed in the system are explained. In each instance, the LaTeX expression representing the algorithm is provided first, and then an explanation is given of its application and functions. This is done to help the reader comprehend both the high-level view and the working of the developed solution.

Partnership Class

A partnership is a representation of the parent pair and their offspring in the family tree. It can also be perceived as a marriage/partnership node which joins the pair of parents to their offspring in one unit.

This gives room to model family relationships in a straightforward manner. Within this class, the parents attribute holds the mother and father of a relationship, and the children attribute holds a list of all the children within a relationship. Along with these relationships, this class is linked to the person class by back-references. The child holds a reference to a relationship (child_backref), and a parent holds a reference to all relationships within which it is a parent (partnership_backrefs). This creates a relationship tree that can be traversed forward and backward efficiently.

Further into the phase of the project, various methods in the partnership class were enabled through the instructor in the context of the project framework. These methods are `__str__`, `__repr__`, `is_parent`, `is_mother`, `is_father`, `add_parent`, `add_child`, `add_children`, and the 'traverse' function. These methods were each incorporated to perform distinct duties in the context of family relationships, to be clarified in the coming sections.

__str__ Function

`__str__` function defines how a `Partnership` object is represented as a string. When the object is printed or converted to a string, it returns the `Partnership`'s name if it exists. If there is no name whatsoever it returns the default value `"Partnership"`.

```
1 Algorithm __str__( partnership ):
2   IF partnership.name IS NOT NULL AND partnership.name IS NOT EMPTY:
3     RETURN partnership.name
4   ELSE:
5     RETURN "Partnership"
```

Figure 1 `__str__` Function at `Partnership` class Line 1: Defines the `__str__` method for a `Partnership` object. This method decides

what string will be shown when the object is printed.

Line 2: Checks whether the `Partnership` has a valid name. If name does exist and is not empty, it will be used.

Line 3: Returns the `Partnership` name as the string representation.

Line 4: The alternative case when the `Partnership` has no name.

Line 5: Returns the default string `"Partnership"` so the object still has a readable output.

__repr__ Function

The `__repr__` function is acting like the `__str__` function in this project, which means it defines how a `Partnership` object is represented whenever it would appear in the console or when embedded into another data structure, such as a list. If the `Partnership` has a name, return the name; else use some default string.

```

1 Algorithm __repr__( partnership ):
2   IF partnership.name IS NOT NULL AND partnership.name IS NOT EMPTY:
3     RETURN partnership.name
4   ELSE:
5     RETURN "Partnership"

```

Figure 2 `__repr__` Function at *Partnership* class Line 1: Definition of the `__repr__` method for a *Partnership* object, which controls its representation.

Line 2: Checks whether the *Partnership* has a valid name.

Line 3: Returns the name of the *Partnership* if it exists.

Line 4: The alternative case when the *Partnership* has no name.

Line 5: Returns the default string "Partnership" to ensure a readable output.

is_parent Function

The `is_parent` function checks if the individual is parent or not. The function returns the value `True` if the individual is mother or father. If the individual is not a parent the code automatically returns the value `False` for code stability in the future usage.

```

1 Algorithm is_parent( partnership, individual ):
2   IF partnership.parents.mother EQUALS individual:
3     RETURN True
4   IF partnership.parents.father EQUALS individual:
5     RETURN True
6   RETURN False

```

Figure 3 is_parent Function at Partnership class Line 1: Definition of the is_parent method for a partnership object, which checks the individual is parent or not.

Line 2: Checks whether the partnership's mother is the individual's mother.

Line 3: Returns true if it is correct.

Line 4: Checks whether the partnership's father is the individual's father.

Line 5: Returns true if it is correct.

Line 6: Returns the default answer as False to ensure stability.

is_mother Function

The is_mother function checks if the individual is a mother or not. The function returns the value True if the individual is a mother. If the individual is not a mother the code automatically returns the value False for code stability in the future usage.

```
1 Algorithm is_mother( partnership, individual):  
2 IF partnership.parents.mother EQUALS individual:  
3     RETURN True  
4     RETURN False
```

Figure 4 is_mother Function at Partnerships class Line 1: Definition of the is_mother method for a partnership object, which checks the individual is mother or not.

Line 2: Checks whether the partnership's mother is the individual's mother.

Line 3: Returns true if the individual is the mother.

Line 4: Returns false if the individual is not a mother.

is_father Function

The is_father function checks if the individual is father or not. The function returns the value True if the individual is father. If the individual is not a father, the code automatically returns the value False for code stability in the future usage.

```

1 Algorithm is_father( partnership, individual ):
2   IF partnership.parents.father EQUALS individual:
3     RETURN True
4   RETURN False

```

Figure 5 is_father Function at Partnership class Line 1: Definition of the is_father method for a partnership and individual object, which checks the individual is father or not.

Line 2: Checks whether the partnership's mother is the individual's father.

Line 3: Returns true if the individual is the father.

Line 4: Returns true if the individual is not the father.

add_parent Function

The add_parent function adds the partnership to the parent list. The function checks all of the parents and calls the add_parent function with the partnership and parent parameters.

```

1 Algorithm add_parent( partnership, parent ):
2   IF parent IS instance of female_person:
3     partnership.parents.mother SET parent
4   ELSE IF parent IS instance of male_person:
5     partnership.parents.father SET parent
6   IF partnership NOT IN parent.partnership_backrefs:
7     APPEND partnership TO parent.partnership_backrefs

```

Figure 6 add_parent Function at Partnership class Line 1: Definition of the is_parent method for a partnership object, which checks the individual is parent or not.

Line 2: Checks whether the partnership's mother is the individual's mother.

Line 3: Returns true if it is correct.

Line 4: Checks whether the partnership's father is the individual's father.

Line 5: Returns true if it is correct.

Line 6: Returns the default answer as False to ensure stability.

add_child Function

The add_child function's main task is to add a child to the partnership. This is done by checking if the child does not exist in the partnership's children. Then adds the child to the partnership. Then sets the child's back reference to the partnership which is very important because in the code we can go through child to partnership as well as with the back reference we can go through partnership to child.

```
1 Algorithm add_child( partnership, child ):
2   IF child NOT IN partnership.children:
3     ADD child TO partnership.children
4     SET child.child_backref TO partnership
```

Figure 7 add_child Function at Partnership class Line 1: Defines the add_child method with partnership and child as an input object.

Line 2: Checks whether the child is the partnership child or not.

Line 3: If the child is not a child of the partnership, the code adds the child to the partnership.

Line 4: If the child is not a child of the partnership, we set the child's back reference to the partnership.

add_children Function

The add_children function's main task is to add multiple children to the partnership. This is done by adding each child in the children list by calling the add_child function.

```

1 Algorithm add_children( partnership, children ):
2   FOR EACH child IN children:
3     CALL add_child( partnership, child )

```

Figure 8 add_children Function at Partnership class Line 1: Defines the add_children method with partnership and child as an input object.

Line 2: For loop gets one child in the children one at a time.

Line 3: Calls the add_child function with the input parameters as partnership and child.

traverse Function

The traverse function is using breadth-first search traversal in the family tree from a given person. The search passes through parents, children, siblings, and relationships such as cousins. Each visited person is recorded along with a level indicating their position relative to each individual in the tree. The search also keeps record of the individual with the lowest level encountered.

```

1 Algorithm traverse( start_person, start_level ):
2   INITIALIZE visited_nodes as empty list
3   INITIALIZE visited_persons as empty set
4   INITIALIZE queue with (start_person, start_level)
5   ADD start_person to visited_persons
6   SET min_level to +infinity
7   SET min_level_node to NULL
8   WHILE queue is not empty:
9     (current_person, level) SET REMOVE first element from queue
10    CREATE visit_node vn with (current_person, level)
11    ADD vn to visited_nodes
12    IF level < min_level:
13      min_level SET level
14      min_level_node SET vn
15    IF visit_callback exists:
16      CALL visit_callback(current_person, level)
17    IF current_person has child_backref:
18      partnership SET current_person.child_backref
19      IF mother exists AND not visited:
20        ADD mother to visited_persons
21        ADD (mother, level - 1) to queue
22      IF father exists AND not visited:
23        ADD father to visited_persons
24        ADD (father, level - 1) to queue

```

```

25  FOR EACH partnership in current_person.partnership_backrefs:
26      FOR EACH child in partnership.children:
27          IF child not visited:
28              ADD child to visited_persons
29              ADD (child, level + 1) to queue
30  IF current_person has child_backref:
31      partnership SET current_person.child_backref
32  FOR EACH sibling in partnership.children:
33      IF sibling ≠ current_person AND not visited:
34          ADD sibling to visited_persons
35          ADD (sibling, level) to queue
36  IF current_person has child_backref:
37      partnership SET current_person.child_backref
38  FOR EACH parent in (mother, father):
39      IF parent exists AND has child_backref:
40          parent_partnership SET parent.child_backref
41  FOR EACH parent_sibling in parent_partnership.children:
42      IF parent_sibling ≠ parent AND not visited:
43          ADD parent_sibling to visited_persons
44          ADD (parent_sibling, level – 1) to queue
45  FOR EACH partnership of parent_sibling:
46      FOR EACH cousin in partnership.children:
47          IF cousin not visited:
48              ADD cousin to visited_persons
49              ADD (cousin, level) to queue

```


Figure 9 *traverse Function at Partition ship class* Line 1: Defines the traverse method with start_partition and start_level as an input.

Line 2: Makes visited_nodes to be an empty list.

Line 3: Makes visited_partitions as an empty set.

Line 4: Makes a queue with the start_partition and start_level.

Line 5: Adds start_partition to the visited_partitions.

Line 6: Sets min_level to +infinity.

Line 7: Sets min_level_node to NULL.

Line 8: While loop that checks if the queue is not empty.

Line 9: Removes the first element from the queue.

Line 10: Creates a visit_node class vn with current_partition and level.

Line 11: Adds vn to the visited_nodes.

Line 12: Checks that if the level is less than min_level.

Line 13: If level is less than min_level, sets level to min_level.

Line 14: If level is less than min_level, sets vn to the min_level_node.

Line 15: Checks if visit_callback exists.

Line 16: If visit_callback exists, calls visit_callback function with current_partition and level.

Line 17: Checks current partition has any child_backref.

Line 18: If current partition has child_backref, sets current_partition's child_backref to partition ship.

Line 19: If current partition has child_backref, checks if mother exists and not visited.

Line 20: If current partition has child_backref and if the mother exists and not visited, adds mother to visited partition.

Line 21: If current partition has child_backref and if the mother exists and not visited, adds mother to the queue with the level of level-1.

Line 22: If current partition has child_backref, checks if the father exists and not visited.

Line 23: If current partition has child_backref and checks if the father exists and not visited, adds father to the visited_partitions.

Line 24: If current_p son has child_backref and checks if the fath exists and not visited, adds fath to the queue with the level of level-1.

Line 25: Creates a loop for each partn ship in current_p son's partn ship_backrefs.

Line 26: Creates a loop for each child in partn ship.children while maintaining the loop for each partn ship in current_p son's partn ship_backrefs.

Line 27: Checks if a child is not visited as well as within a loop for each child in partn ship.children while maintaining the loop for each partn ship in current_p son's partn ship_backrefs.

Line 28: If a child is not visited as well as within a loop for each child in partn ship.children while maintaining the loop for each partn ship in current_p son's partn ship_backrefs, adds child to the visited_p sons

Line 29: If a child is not visited as well as within a loop for each child in partn ship.children while maintaining the loop for each partn ship in current_p son's partn ship_backrefs, adds child to the queue with the level of level -1.

Line 30: Checks if a current_p son has a child_backref

Line 31: If a current_p son has a child_backref, sets current_p son's child_backref to partn ship.

Line 32: If a current_p son has a child_backref while maintaining the loop for each partn ship in current_p son's partn ship_backrefs, makes a loop for each sibling in partn ship's children.

Line 33: If a current_p son has a child_backref with a loop for each sibling in partn ship's children, checks if the sibling is not equal to the current_p son and not visited.

Line 34: If a current_p son has a child_backref with a loop for each sibling in partn ship's children as well as with the sibling is not equal to the current_p son and not visited, adds sibling to visited_p son.

Line 35: If a current_p son has a child_backref with a loop for each sibling in partn ship's children as well as with the sibling is not equal to the current_p son and not visited, adds sibling to the queue with the level of level.

Line 36: Checks if a current_p son has a child_backref.

Line 37: If a current_p son has a child_backref, set current_p son's child_backref to partn ship.

Line 38: If a current_p son has a child_backref, makes a loop for each parent in moth and fath .

Line 39: If a current_p son has a child_backref with a loop for each parent in moth and fath , checks if parent exists and has a child_backref.

Line 40: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` and parent exists and has a `child_backref`, sets parent's `child_backref` to `parent_partn_ship`.

Line 41: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref`, creates a loop for each `parent_sibling` in `parent_partn_ship`'s children.

Line 42: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children, checks if `parent_sibling` is not equal to parent and not visited.

Line 43: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children and if `parent_sibling` is not equal to parent and not visited, adds `parent_sibling` to `visited_p_sons`.

Line 44: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children and if `parent_sibling` is not equal to parent and not visited, adds `parent_sibling` to a queue with the level of level -1.

Line 45: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children, makes a loop for each `partn_ship` in `parent_sibling`.

Line 46: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children, makes a loop for each `partn_ship` in `parent_sibling`.

Line 47: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children and fulfills the loop for each `partn_ship` in `parent_sibling`, checks if cousin is not visited.

Line 48: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children and fulfills the loop for each `partn_ship` in `parent_sibling` and if cousin is not visited, adds cousin to `visited_p_sons`.

Line 49: If a current_`_p` son has a `child_backref` with a loop for each parent in `moth` and `fath` as well as if parent exists and has a `child_backref` while satisfying a loop for each `parent_sibling` in `parent_partn_ship`'s children and

fulfills the loop for each partn ship in parent_sibling and if cousin is not visited, adds cousin to a queue with the level of level.

Line 50: Checks if level sorting is enabled.

Line 51: If level sorting is enabled, sorts visited_nodes by level.

Line 52: Returns visited_nodes and min_level_node.

2) Person Class

The Person class represents an individual in the family tree. Each person can be assigned a name and an optional DNA code, and can solve extra information using a flexible properties dictionary. The class is designed to support family relationship queries by linking individuals to partnerships through back-references.

A person might be connected to the family tree in to different roles. Firstly a person can be a child of a certain partnership using child_bacref which points to the partnership where the person appears as child. Secondly one can be parent in one or more partnerships which is realized by partnership_backrefs storing a list of partnerships where this person is registered as parent. These links make it possible to move upwards to parents and downwards to children when navigating a family tree.

In the project framework, the Person class includes a number of utility and relationship related methods. While some have been implemented by our team with the required functionality, others were provided as part of the base structure of this project. In what follows we focus on these methods provided by our group. The methods implemented by our team and explained in this reports are: `__str__`, `__eq__`, `ancestor` and `descendant`.

`__str__` Function

The '`__str__`' function returns a string representation of a person if a person has a name that's what returned otherwise "Unknown" is set as a default output.

```
1 Algorithm __str__( person ):
2 IF person.name IS NOT NULL AND person.name IS NOT EMPTY:
3     RETURN person.name
4 ELSE:
5     RETURN "Unknown"
```

Figure 10 `__str__` Function at Person class Line 1: Defines the method `__str__` which decides how the person object will be represented as a string or printed out.

Line 2: It checks if the individual has a valid name.

Line 3: This returns the person's name if it is available.

Line 4: Starts the alternative case if the name does not exist.

Line 5: This is a return statement with the default value "Unknown" to make sure the object is always printable.

`__eq__` Function

The "`__eq__`" method is used to compare the equality of the objects of the person class. Two persons are said to be equal if both are of the same class and have the same name. The method will return False if the object passed is not a person object.

```
1 Algorithm __eq__( self, other ):
2 IF other IS NOT instance of person:
3     RETURN False
4 IF self.name EQUALS other.name:
5     RETURN True
6 ELSE:
7     RETURN False
```

Figure 11 `__eq__` Function at Person class Line 1: Defines the "`__eq__`" method for comparing the current person with another object.

Line 2: Checks whether the other object is an instance of the person class.

Line 3: If the other object is an instance of the person class, this line will return false if the passed object is not a person.

Line 4: Checks if the names of both person objects are equal.

Line 5: If the names of both person objects are equal, returns true.

Line 6: Checks if the names of both person objects are not equal.

Line 7: If the names of both person objects are not equal, it will return false if the names don't match.

ancestor Function

The ancestor function returns a list that includes all ancestors of a dedicated person traveling upward through the family tree. The parent links are traversed based on child_backref point as well as mother and father at different levels. The visited set is used to avoid returning a person or processing it to avoid loops.

```

1 Algorithm ancestors( person ):
2   INITIALIZE ancestors_list as empty list
3   INITIALIZE visited as empty set
4   DEFINE recursive function collect( current_person ):
5     IF current_person IN visited:
6       ADD current_person TO visited
7       RETURN current_person
8     IF current_person.child_backref IS NOT NULL:
9       partnership SET current_person.child_backref
10      IF partnership.parents.mother IS NOT NULL
11        AND mother NOT IN visited:
12          ADD mother TO ancestors_list
13          CALL collect( mother )
14      IF partnership.parents.father IS NOT NULL
15        AND father NOT IN visited:
16          ADD father TO ancestors_list
17          CALL collect( father )
18  CALL collect( person )
19  RETURN ancestors_list

```

Figure 12 ancestor Function at Person class Line 1: Begins the ancestor method for a person

Line 2: Initializes ancestors_list as an empty list.

Line 3: Initializes visited as an empty set.

Line 4: Defines recursive function as collect function with an input of cur-

rent_p son.

Line 5: Defines recursive function as collect function with an input of current_p son and checks if current_p son is in visited.

Line 6: Defines recursive function as collect function with an input of current_p son and adds current_p son to visited.

Line 7: Defines recursive function as collect function with an input of current_p son and if current_p son is in visited, returns current p son.

Line 8: Defines recursive function as collect function with an input of current_p son and checks if current_p son's child_backref is not null value.

Line 9: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value, sets current p son's child_backref to partn ship.

Line 10: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value, checks if partn ship's parent's moth is not null and moth is not visited.

Line 11: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value and if partn ship's parent's moth is not null, checks moth is not visited.

Line 12: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value and if partn ship's parent's moth is not null and moth is not visited, adds moth to ancestors_list.

Line 13: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value and if partn ship's parent's moth is not null and moth is not visited, calls collect function with the input of moth .

Line 14: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value, checks if partn ship's parents's fath is not null and fath is not in visited.

Line 15: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value and if partn ship's parents's fath is not null, checks fath is not in visited.

Line 16: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value and if partn ship's parents's fath is not null and fath is not in visited, adds fath to ancestors_list.

Line 17: Defines recursive function as collect function with an input of current_p son and if current_p son's child_backref is not null value and if

partnership's parents's father is not null and father is not in visited, calls collect function with the input of father.

Line 18:Calls collect function with the input of person.

Line 19:Returns the ancestors_list.

descendant Function

The descendant method on the other hand traverses down a family tree to return all descendants of a person. The method iterates over all pairing that involve a person as a parent and append all their child to a set of descendants. The method employs a visited set to ensure that a person is not traversed more than once. The method is recursive meaning that it calls itself to move down generations.

```
1 Algorithm descendants( person ):
2   INITIALIZE descendants_list as empty list
3   INITIALIZE visited as empty set
4   DEFINE recursive function collect( current_person ):
5     IF current_person IN visited:
6       RETURN current_person
7     ADD current_person TO visited
8     FOR EACH partnership IN current_person.partnership_backrefs:
9       FOR EACH child IN partnership.children:
10        IF child NOT IN visited:
11          ADD child TO descendants_list
12          CALL collect( child )
13    CALL collect( person )
14    RETURN descendants_list
```

Figure 13 descendants Function at P son class Line 1: Begins the descendants method for a p son

Line 2: Initializes descendants_list as an empty list.

Line 3:Initializes visited as an empty set.

Line 4:Defines recursive function as collect function with an input of current_p son.

Line 5:Defines recursive function as collect function with an input of current_p son and checks if current_p son is in visited.

Line 6:Defines recursive function as collect function with an input of current_p son and adds current_p son to visited.

Line 7:Defines recursive function as collect function with an input of current_p son and if current_p son is in visited, returns current p son.

Line 8:Defines recursive function as collect function with an input of current_p son and makes a loop for each partn ship in current_p son's partn ship_backrefs.

Line 9:Defines recursive function as collect function with an input of current_p son while satisfying a loop for each partn ship in current_p son's partn ship_backrefs, creates a loop for each child in partn ship's children.

Line 10:Defines recursive function as collect function with an input of current_p son while satisfying a loop for each partn ship in current_p son's partn ship_backrefs and fulfills a loop for each child in partn ship's children, checks if the child is not in visited.

Line 11:Defines recursive function as collect function with an input of current_p son while satisfying a loop for each partn ship in current_p son's partn ship_backrefs and fulfills a loop for each child in partn ship's children and if the child is not in visited, adds child to descendants_list.

Line 12:Defines recursive function as collect function with an input of current_p son while satisfying a loop for each partn ship in current_p son's partn ship_backrefs and fulfills a loop for each child in partn ship's children and if the child is not in visited, calls collect function with the input of child.

Line 13: Calls collect function with the input of p son.

Line 14: Returns descendats_list.

6. RESULTS

The system was tested with a test: The test case is called as “kinship relation tests” which has 7 tests within it: “test_unrelated_xx16_xy24”, which is a test for two people if they are related or not. “test_unrelated_xx19_xx18”, which is a test for two people if they are related or not. “test_first_cousins_xy22_xx18”, which is a test for two people if they are first cousins or not. “test_first_cousins_once_removed_xy24_xx18”, which is a test for two people if they are first cousins who are removed or not. “test_greatuncle_greatnephew_xy24_xy15”, which is a test for two people if they are greatnephew/greatuncle or not. “test_uncle_niece_xx46_xy15” which is a test for two people if they are uncle/niece or not. and “test_no_such_p son_ror” which is a test for a p son if p son is real or not. In Figure 14, after completing the “kinship tests”, it can be observed that all tests are completed without any errors which means that the first test’s result is that two people are unrelated. For the second test’s result is that two people are unrelated. For the third test’s results, two people are first cousins. For the fourth test’s result, two people are first cousins are once removed. For the fifth test’s result two people are greatuncle or greatnephew. For the sixth test’s results, two people are uncle or niece. For the seventh test’s result, the p son is a real p son.

After this step the program runs a simulation and gives the results called “Relationship Analysis Results”. In these results it can be observed that p son T19_F1_XX16 and T19_F1_XY24 are unrelated, p son T19_F1_XX19 and T19_F1_XX18 are unrelated, p son T19_F1_XY22 and T19_F1_XX18 are first_cousins, p son T19_F1_XY24 and T19_F1_XX18 are first_cousins_once_removed, p son T19_F1_XY24 and T19_F1_XY15 are greatuncle_greatnephew, p son T19_F1_XX46 and T19_F1_XY15 are uncle_niece. After these results, we observe that the program runs successfully.

```

conda activate base
/opt/anaconda3/bin/python "/Users/huseyinserkangunyol/Desktop/cng465 project/test_kinship.py"
● (base) huseyinserkangunyol@Huseyins-MacBook-Pro-5 cng465 project % conda activate base
● (base) huseyinserkangunyol@Huseyins-MacBook-Pro-5 cng465 project % /opt/anaconda3/bin/python "/Users/huseyinserkangunyol/Desktop/cng465 project/test_kinship.py"
Running kinship relationship tests...
[PASS] test_unrelated_xx16_xy24
[PASS] test_unrelated_xx19_xx18
[PASS] test_first_cousins_xy22_xx18
[PASS] test_first_cousins_once_removed_xy24_xx18
[PASS] test_greatuncle_greatnephew_xy24_xy15
[PASS] test_uncle_niece_xx46_xy15
[PASS] test_no_such_person_error

All tests completed!
● (base) huseyinserkangunyol@Huseyins-MacBook-Pro-5 cng465 project % /opt/anaconda3/bin/python "/Users/huseyinserkangunyol/Desktop/cng465 project/run_example.py"
=====
Forensic DNA Kinship Project - Phase 1
Relationship Analysis Results
=====

[OK] T19_F1_XX16      <=> T19_F1_XY24      : unrelated
[OK] T19_F1_XX19      <=> T19_F1_XX18      : unrelated
[OK] T19_F1_XY22      <=> T19_F1_XX18      : first_cousins
[OK] T19_F1_XY24      <=> T19_F1_XX18      : first_cousins_once_removed
[OK] T19_F1_XY24      <=> T19_F1_XY15      : greatuncle_greatnephew
[OK] T19_F1_XX46      <=> T19_F1_XY15      : uncle_niece

=====
TEST Completed!
=====
○ (base) huseyinserkangunyol@Huseyins-MacBook-Pro-5 cng465 project % █

```

Figure 14 Test Results

7.DATA FLOW DIAGRAM

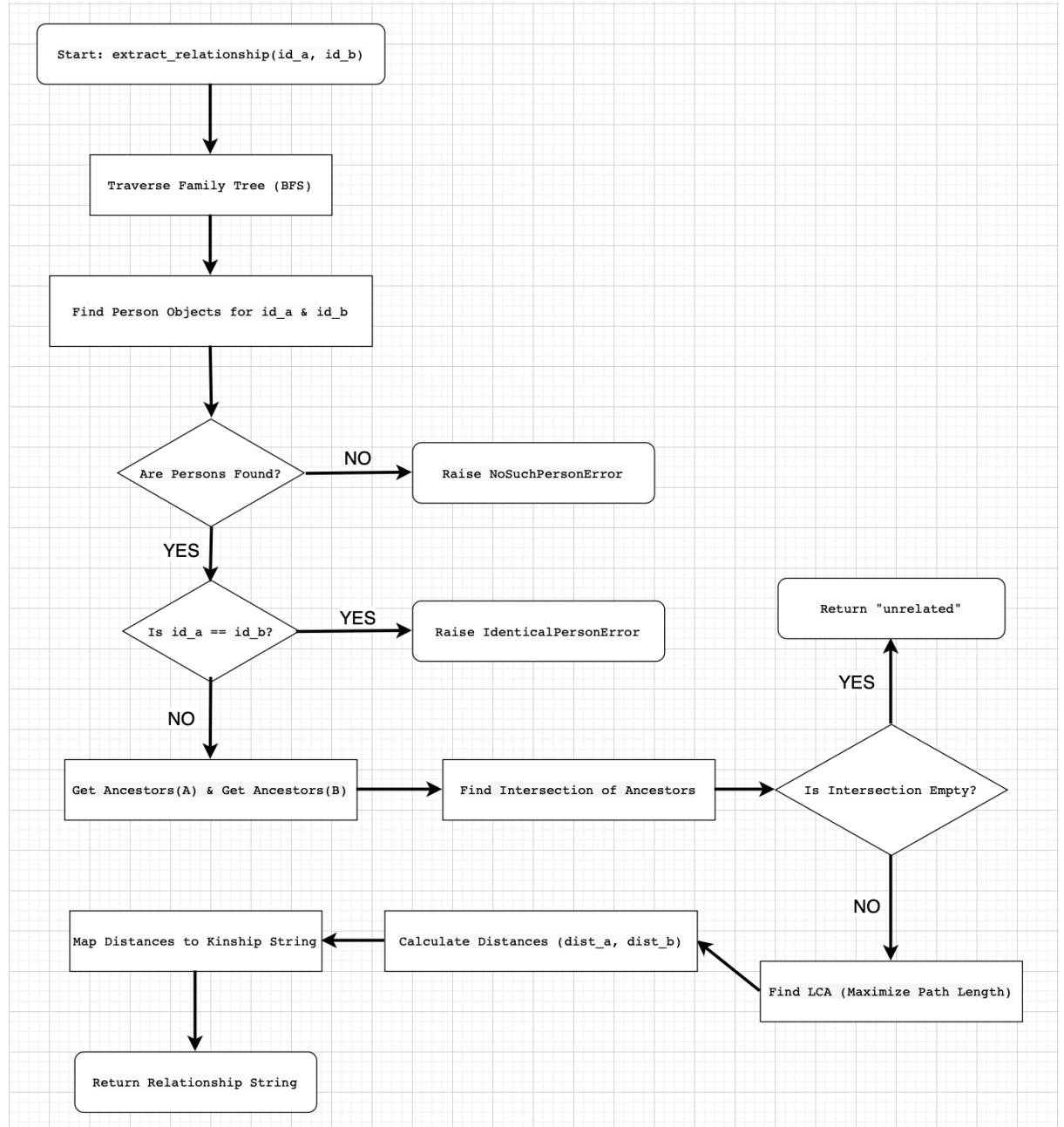


Figure 15 Data Flow Diagram Figure 15 represents the digitally drawn Data Flow Diagram of the system.

8. CONCLUSION

In conclusion, The objective of this project was to develop a basic family tree structure with analysis of knipship relations using Python. With using both person and partnership classes a basic structure was created to make it possible two way traversal in parent child relationship with using back references. Multiple functions were implemented to make it possible this structure including functions for identifying ancestors and descendants and tracing paths to visited people.

Upon this framework the logic for performing a knipship relationship was built using lowest common ancestor search and distance calculations. The system was able to detect basic relationships between cousins, uncles etc. From both the example script and the unit tests it was confirmed that everything is working flawlessly based on the given family arrangement. All test are passing which validates that the system is also able to calculate different relationships as well as handle with errors by throwing exceptions

Although the project deals with the most general cases as outlined by the project tasks there is room for including the system to involve even more complex family relationships. Still the completion of the task meets the requirements of the project and serves as the initial stage of analyzing kinship by means of the symbolic family tree.

9. REFERENCES

N/A